ENTREGA CONTÍNUA DE UMA PLATAFORMA DE PEDIDOS EM MICROSSERVIÇOS

Parte Teórica: Do Docker Compose ao Kubernetes com Observabilidade e CI/CD

Caso: Loja Veloz (Pedidos Veloz) | Disciplina: Arquitetura Cloud DevOps & Containers

1. Contexto do Problema (Lide) e Arquitetura de Microsserviços

Nos últimos 12 meses, o e-commerce Loja Veloz cresceu bastante, mas a parte técnica não acompanhou esse ritmo. Na prática, cada desenvolvedor subia os serviços 'como dava' na própria máquina, as atualizações em produção geravam quedas frequentes no site e, quando um pedido travava, ninguém sabia ao certo se a falha tinha acontecido no Gateway, no banco de dados, na reserva de estoque ou na operadora de pagamento. Como a empresa terá uma grande campanha promocional em 4 semanas, estruturei este plano Cloud DevOps para organizar o fluxo do ambiente local até a produção.

A plataforma 'Pedidos Veloz' foi pensada seguindo a arquitetura de microsserviços. Em vez de manter um sistema monolítico (onde todo o código fica preso em um único programa gigante e qualquer erro derruba a loja inteira), dividimos a aplicação em quatro serviços independentes que conversam via HTTP e eventos: API Gateway, Serviço de Pedidos, Serviço de Estoque e Serviço de Pagamentos, apoiados pelo PostgreSQL e pelo Redis. Assim, cada serviço tem sua própria responsabilidade e pode ser atualizado ou escalado separadamente.

Nesse cenário cloud-native, o papel do DevOps vai muito além de apenas escrever scripts: trata-se de unir os times de desenvolvimento (Devs) e operações (SRE/Ops) por meio de automação, padronização e telemetria. Em vez de entregas manuais e arriscadas, o DevOps garante que qualquer código novo passe por testes automáticos, vire uma imagem segura e chegue em produção sem tirar o sistema do ar.

2. Conteinerização: Diferenças entre Docker e Kubernetes

Durante os estudos da disciplina, fica claro que a conteinerização foi o que viabilizou os microsserviços modernos. Um container empacota o código da aplicação junto com suas bibliotecas e configurações em um processo isolado e leve, acabando com o clássico problema do 'na minha máquina funciona, mas no servidor não'.

Muita gente confunde o papel do Docker com o do Kubernetes, mas eles atuam em etapas diferentes e complementares. O Docker (junto com o Docker Compose) é voltado para construir a imagem do container e rodar múltiplos containers em uma única máquina, sendo perfeito para padronizar o ambiente local dos desenvolvedores. Já o Kubernetes é um orquestrador de containers para produção: ele gerencia dezenas ou centenas de containers distribuídos em vários servidores (nós).

Critério / Aspecto	Docker & Docker Compose	Kubernetes (K8s)
Foco Principal	Construção de imagens e execução local em um único host.	Orquestração distribuída em cluster de múltiplos servidores.
Quando Utilizar	Desenvolvimento diário, testes locais e validação rápida.	Ambientes de produção e homologação com alta disponibilidade.
Tolerância a Falhas	Básica (reinicia o container se o processo cair na máquina).	Auto-cura completa (recria Pods em outro servidor se um nó falhar).
Escalabilidade	Manual (limitada aos recursos de CPU/RAM da máquina local).	Automática via HPA/VPA de acordo com o pico de acessos.
Atualização (Deploy)	Recria os containers locais (gera pequena parada).	Rolling Update, Blue-Green ou Canary sem indisponibilidade.

3. Fundamentação Teórica: Orquestração, CI/CD e Observabilidade

3.1 Orquestração de Containers em Produção

Quando temos quatro ou mais microsserviços rodando várias cópias ao mesmo tempo, fica impossível gerenciar tudo na mão. A orquestração de containers com Kubernetes funciona baseada em 'estado desejado' (declarado em arquivos YAML). Se definirmos que o Serviço de Pedidos precisa de 3 réplicas saudáveis, o Control Plane do Kubernetes monitora os Pods o tempo todo através de sondas (livenessProbe para saber se o container travou e readinessProbe para saber se ele já pode receber clientes), distribuindo o tráfego de rede automaticamente através dos objetos Service.

3.2 CI/CD em Ambientes Distribuídos

Em uma arquitetura distribuída, fazer deploy manual copiando arquivos para o servidor é receita certa para erros. Por isso adotamos CI/CD (Integração Contínua e Entrega Contínua). Na etapa de CI, cada alteração enviada ao repositório dispara automaticamente a validação de sintaxe (lint), os testes automatizados de cada microsserviço, a construção da imagem Docker e a checagem de segurança (scan de vulnerabilidades). Se tudo passar nos 'gates' de qualidade, a etapa de CD publica a imagem versionada no Registry e atualiza o cluster Kubernetes de forma controlada.

3.3 Os Três Pilares da Observabilidade (Métricas, Logs e Traces)

Monitorar apenas se o servidor está ligado não basta em microsserviços. A observabilidade nos permite entender o que acontece dentro do sistema através de três pilares que trabalham juntos:

- **1. Métricas (Prometheus & Grafana):** São valores numéricos agregados ao longo do tempo, como uso de CPU, quantidade de pedidos por segundo e taxa de erros HTTP 5xx (os chamados Golden Signals). Elas respondem rapidamente 'que' algo está fora do normal.
- **2. Logs Estruturados (JSON no stdout):** São os registros detalhados de cada evento que aconteceu no código (ex: 'Estoque insuficiente para o SKU X'). Seguindo o 12-Factor App, os containers jogam os logs na saída padrão em formato JSON para facilitar a busca.
- **3. Tracing Distribuído (OpenTelemetry & Jaeger):** É o pilar que resolve a maior dor da Loja Veloz. Quando o cliente clica em comprar, o API Gateway cria um código único (traceId) e o repassa nos cabeçalhos HTTP para Pedidos, Estoque e Pagamentos. Assim conseguimos ver uma linha do tempo completa (spans) mostrando exatamente quantos milissegundos cada microsserviço gastou.

4. Pesquisa de Caso Concreto e Comparativo de Deploy

Caso de Referência Pesquisado: Google Cloud 'Online Boutique' (microservices-demo) & 12-Factor App

Para desenhar a solução da Loja Veloz, pesquisei o repositório oficial de referência 'Online Boutique' (GoogleCloudPlatform/microservices-demo), que é uma loja virtual cloud-native completa em microsserviços rodando no Kubernetes. Assim como no case do Google, adotei um API Gateway na borda, microsserviços desacoplados sem estado (stateless), sondas de saúde nativas, instrumentação com OpenTelemetry/Jaeger e manifestos declarativos com HPA. Também apliquei os princípios da metodologia 12-Factor App (fator III - configurações em variáveis de ambiente via ConfigMap/Secret; fator VI - processos stateless; e fator XI - logs como fluxos de eventos).

4.1 Comparação entre Estratégias de Deploy em Produção

Estratégia	Como Funciona na Prática	Custo de Infra	Uso na Loja Veloz
Rolling Update	Substitui os Pods antigos pelos novos aos poucos (maxUnavailable: 0).	Baixo (apenas +1 Pod extra durante a troca).	Padrão adotado em todos os microsserviços no MVP.
Blue-Green	Sobe um ambiente inteiro duplicado (Verde) e vira a chave do roteador.	Alto (exige o dobro de servidores ativos).	Descartado no MVP pelo custo elevado de manter 2 ambientes.
Canary Release	Libera a versão nova para apenas 5% a 10% dos clientes inicialmente.	Baixo (controlado via Ingress ou Service Mesh).	Recomendado para mudanças críticas no pagamentos-service.

5. Justificativa das Decisões Arquiteturais Adotadas no Projeto

Todas as escolhas técnicas feitas para o MVP da Loja Veloz buscaram equilibrar simplicidade de manutenção (já que o prazo é de 4 semanas até a campanha promocional) com segurança e confiabilidade de nível de produção:

- **Padronização Local com Docker Compose:** Escolhi o Docker Compose para o ambiente de desenvolvimento porque ele permite subir os 4 microsserviços, o PostgreSQL, o Redis e toda a pilha de observabilidade (Jaeger, Prometheus e Grafana) com um único comando (docker compose up -d --build). Isso elimina as diferenças de configuração entre as máquinas dos desenvolvedores.
- **Imagens Multi-Stage e Usuário Não-Root:** Nos Dockerfiles, utilizei multi-stage build com a imagem base enxuta node:20-alpine3.19. O primeiro estágio roda os testes e valida o código; o segundo copia apenas o código de produção e executa com um usuário sem privilégios (UID 10001). Isso reduz o tamanho da imagem e impede que um invasor ganhe acesso root no servidor.
- **Mensageria Assíncrona (Evento PedidoCriado):** Optei por incluir o Redis junto ao padrão Outbox para publicar o evento 'PedidoCriado' assim que o pedido é pago. Isso desacopla o fluxo principal de ações secundárias (como envio de nota fiscal ou e-mail), deixando a resposta de compra muito mais rápida para o cliente.
- **Estratégia de Deploy RollingUpdate (Zero-Downtime):** Entre Rolling Update, Blue-Green e Canary, escolhi o RollingUpdate configurado com maxUnavailable: 0 e maxSurge: 1 como padrão do MVP. Ele cria primeiro o Pod novo, aguarda a readinessProbe confirmar que o serviço está pronto e só então desliga o Pod antigo. Assim eliminamos as quedas durante deploys sem precisar dobrar o custo de infraestrutura como no Blue-Green.
- **Escalabilidade com HPA (em vez de VPA isolado):** Como os nossos microsserviços foram construídos de forma stateless (sem guardar sessão na memória local), o Horizontal Pod Autoscaler (HPA) é a escolha ideal para o pico da campanha promocional: quando a CPU passa de 70%, o Kubernetes cria novas réplicas em segundos. O VPA precisaria reiniciar os Pods para alterar memória/CPU, o que atrapalharia durante um pico de acessos.
- **Infraestrutura como Código (IaC) com Terraform:** Para garantir que o ambiente de nuvem (VPC isolada, cluster Kubernetes EKS e banco RDS PostgreSQL Multi-AZ) possa ser recriado de forma idêntica e auditável, estruturei os arquivos do Terraform parametrizados por variáveis.

Camada / Requisito	Tecnologia Escolhida	Benefício Direto para a Campanha da Loja Veloz
Ambiente Local	Docker Compose + .env (12-Factor)	Time inteiro sobe os 9 containers do sistema com um único comando.
Orquestração & Escala	Kubernetes + HPA v2 (CPU 70% / RAM 80%)	Escala de 3 para 12 réplicas em segundos durante picos de tráfego.
Segurança (DevSecOps)	Alpine Multi-Stage + PSA Restricted + Trivy	Containers sem root (UID 10001) e imagens bloqueadas se houver CVEs.
Observabilidade	OpenTelemetry + Jaeger + Prometheus	Rastreio exato de falhas entre serviços usando o mesmo traceId.

6. Referências Bibliográficas (Fontes Primárias)

1. KUBERNETES OFFICIAL DOCUMENTATION. Concepts: Workloads (Pods, Deployments), Services, ConfigMaps, Secrets, Pod Security Admission & Horizontal Pod Autoscaling. Disponível em: https://kubernetes.io/docs/home/

2. DOCKER OFFICIAL DOCUMENTATION. Multi-stage builds, Dockerfile best practices & Docker Compose specification. Disponível em: https://docs.docker.com/

3. WIGGINS, Adam. The Twelve-Factor App (Metodologia Cloud-Native: Config, Stateless Processes, Disposability, Logs). Disponível em: https://12factor.net/pt_br/

4. GOOGLE CLOUD PLATFORM. Online Boutique (microservices-demo): Cloud-first microservices reference architecture on Kubernetes with OpenTelemetry. Disponível em: https://github.com/GoogleCloudPlatform/microservices-demo

5. OPENTELEMETRY & JAEGER DOCUMENTATION. Context Propagation (W3C Trace Context) and Distributed Tracing. Disponível em: https://opentelemetry.io/docs/

6. HASHICORP TERRAFORM DOCUMENTATION. Infrastructure as Code (IaC), Providers, Modules and State Management. Disponível em: https://developer.hashicorp.com/terraform/docs